Queen's University School of Computing

Interactive Planning Agent

Discord Bot

Adam Neto and Jawad Ahmed

CISC 499 Capstone Project

Professor Christian Muise

Supervisor: Marcus Tantakoun

April 5, 2026

Completed under the Mu Lab

Contents

1 Introduction **3**

2 Problem Background **4**

2.1 AI Planning and PDDL 4

2.2 Human-in-the-Loop Planning 4

2.3 Language Models and Language-to-Plan 5

2.4 Model Context Protocol 6

3 System Overview **7**

3.1 Overall Architecture 7

3.2 `IPA-Discord-Bot` 7

3.3 `planning-as-a-service` 7

3.4 `l2p_mcp` 8

3.5 Interactions Between Repositories 8

4 Approach and Implementation **9**

4.1 Design Approach 9

4.2 Work in `planning-as-a-service` 9

4.3 Work in `l2p_mcp` 10

4.4 Work in `IPA-Discord-Bot` 10

4.5 Deployment and Infrastructure 11

4.6 Key Challenges and Design Tradeoffs 12

5 Results **12**

5.1 Completed Functionality 12

5.2 Demonstration Scenarios 13

5.3 Testing and Validation 13

5.4 Assessment Against Objectives 14

6 Open Problems and Future Work **14**

6.1 Current Limitations 14

6.2 Future Extensions 15

7 Conclusion **16**

A Appendix: Project Repositories **17**

A.1 Foundation Repositories 17

A.2 Repositories Created for this Project 17

B Appendix: Discord Bot Commands **18**

B.1 Planning Commands 18

B.2 Human-in-the-Loop Editing Commands 18

B.3 Validation Commands 18

B.4 MCP Tool Inspection Commands 19

B.5 Chat and Thread Commands 19

B.6 Bot Setting Commands 19

C Appendix: MCP Tools **19**

C.1 `planning-as-a-service` Tools 19

C.2 `L2P` Tools on `l2p_mcp` 20

D Appendix: Supplementary Materials **20**

D.1 Video Demonstration 20

D.2 `IPA-Discord-Bot` Poster 21

1 Introduction

The *Interactive Planning Agent* project explores how large language models, classical AI planning tools, and human-in-the-loop interaction can be combined into a practical planning assistant. Automated planning systems are powerful, but they are often difficult to access directly, especially for users who are not comfortable writing formal planning models by hand. In their survey *LLMs as Planning Formalizers*, Tantakoun et al. (2025) argue that LLMs are often more useful for extracting and refining planning models than for replacing formal planners outright. This project follows that same general direction by making planning workflows more accessible through natural-language interaction, iterative refinement of PDDL domains and problems, and access to solver and validation services.

More specifically, the project focuses on building an interactive planning assistant around a Discord bot. The bot acts as a conversational entry point where users can discuss a planning problem, request plan generation, and iteratively revise domain or problem information. This direction is similar in spirit to recent systems such as *Planning Copilot*, which Benyamin et al. (2025) describe as a chatbot that invokes planning tools through natural-language instructions, and *PDDL-GenAI*, which Wang et al. (2025) present as a tool for lowering the barrier to PDDL modeling inside an existing planning environment. The Interactive Planning Agent follows a comparable motivation, but places that workflow inside Discord and connects it to separate MCP-based services for model editing, solving, and validation.

The final project work was distributed across three closely related repositories. The first repository, `IPA-Discord-Bot`, contains the user-facing planning assistant. It manages Discord interactions, persistent user context, model selection, and the MCP client layer used to communicate with planning, validation, syntax-checking, and PDDL-generating back ends. The second repository, `planning-as-a-service`, provides the solver and validation infrastructure. With this project, those services were also exposed through an MCP server that could be accessed by the Discord bot and other MCP clients. The third repository, `l2p_mcp`, is a standalone MCP server that wraps deterministic portions of the external `L2P` toolkit. Its purpose is to support structured updates to planning domains and tasks so that users can incrementally refine PDDL artifacts instead of restarting the modeling process from scratch.

Taken together, these repositories form a layered system: Discord provides the interface, MCP provides the integration boundary, the `L2P`-based service supports iterative modeling, and `planning-as-a-service` supplies solver-oriented back-end capabilities. This separation of responsibilities was important because it allowed conversational interaction, planning-model editing, and solver access to evolve as related but distinct components.

2 Problem Background

2.1 AI Planning and PDDL

In the survey paper *LLMs as Planning Formalizers*, Tantakoun et al. (2025) describe automated planning as the task of generating action sequences that transform an initial state into a goal state under explicit model constraints. Their survey treats planning problems in terms of states, actions, transitions, an initial state, and a goal. A valid plan is therefore a sequence of actions that remains applicable throughout execution and reaches the goal state from the initial state. Benyamin et al. (2025) make a similar distinction in *Toward PDDL Planning Copilot* while motivating the use of formal planning tools for long-horizon tasks.

Tantakoun et al. (2025) also identify *Planning Domain Definition Language* (PDDL) as one of the central representations used for formal planning tasks. In PDDL, a planning task is separated into a domain file and a problem file. The domain file captures reusable structure such as predicates and actions, while the problem file specifies a concrete instance through its objects, initial state, and goal state. This separation allows one domain model to be reused across multiple tasks by changing only the problem instance. Benyamin et al. (2025) likewise describe planning tasks in terms of paired domain and problem components.

PDDL is central to the Interactive Planning Agent. The Discord bot, the `l2p_mcp`-based modeling workflow, and the planning back ends all operate around the creation, editing, validation, and solving of PDDL artifacts. Following the perspective of Tantakoun et al. (2025), the role of language models in this kind of system is not necessarily to replace formal planners, but to help construct and refine the planning models that those planners depend on.

2.2 Human-in-the-Loop Planning

Human-in-the-loop (HITL) planning is a feature that works to overcome the ambiguity and flaws of real planning task descriptions. Regardless of the specifications of the given planning instructions, there is often a discrepancy between those instructions and the fully structured information required by the planner. Naturally, it is not expected that the user will provide a flawless description of a task, especially given increasing levels of complexity. Important details may be missing, assumptions may be incorrect, and the resulting problem and/or domain models may differ from the user's intent. Such inconsistencies can lead to invalid plans, unsolvable tasks, or technically correct plans that still prove unhelpful. For this reason, HITL was implemented to allow iterative construction and refinement of planning artifacts rather than relying on a single automatic translation step.

This matters especially in PDDL-based systems. Planning models depend on precise predicates, action definitions, objects, and related structures, to the point that even small modeling errors can

lead to invalid or unrealistic plans. Leaving out an initial fact may render the goal unreachable, while forgetting a precondition may allow unrealistic actions. While automation aids in improving efficiency and consistency, the user best understands the real-world task expected from the model. HITL is therefore significant because it provides semantic checks from the user on top of formal validation. In this collaboration, the planner works to generate a plan that is valid with respect to the model, whereas the user has the final say in confirming whether the model captures the intended task accurately. This connects closely to *PDDL-GenAI*: Wang et al. (2025) describe their plugin as addressing not just the generation of PDDL from natural-language descriptions, but also whether the generated models remain semantically aligned with the user's description and solvable by downstream planners. In that sense, both *PDDL-GenAI* and the Interactive Planning Agent treat user input and iterative correction as important parts of making language-assisted planning workflows practically useful.

2.3 Language Models and Language-to-Plan

Language models are useful in this project because they help bridge the gap between natural language requests and the structured representations required by planning tools. A user will typically think of a task in natural language, whether that means moving objects, scheduling activities, or satisfying certain goals under constraints. A planner, however, requires much more explicit information, such as actions, predicates, objects, initial states, and desired outcomes.

This translation can happen at different levels. A model may use a user's input to propose an initial domain and problem, or it may assist with narrower revisions such as adding an object, changing a goal, or refining an action description. Tantakoun et al. (2025) explicitly frame LLMs as modelers that construct and refine planning models for downstream planners. Once those structured artifacts are produced, planners and validators can operate on them more reliably than a language model alone. Benyamin et al. (2025) describe a similar division of labor in *Planning Copilot*, where a chatbot accesses external planning tools for syntax checking, planner selection, plan generation, validation, and simulation.

For this project, language-to-plan was not treated as a single free-form generation step, but instead as a staged process. The bot uses an LLM to convert a natural-language request into structured update payloads which are then sent to planning services through MCP. These payloads are separated into domain-oriented and task-oriented updates, and are handled by `l2p_mcp` to produce the corresponding PDDL artifacts. This reduces the amount of unconstrained text generation involved in the workflow and gives the process a more reliable structure. In this way, the model still performs the interpretation and translation, but its outputs are constrained by the expected tool interfaces rather than being treated as arbitrary text.

This separation is important because it reflects the practical role of LLMs in planning systems. The

model contributes flexibility, natural-language understanding, and the ability to interpret requests that may be underspecified, while the structured planning tools contribute determinism, validation, and solver execution. In the Interactive Planning Agent, language models therefore serve as the front end of the planning workflow. They help users express planning tasks in a more accessible way and support iterative refinement, while the formal planning back ends remain responsible for syntax checking, validation, and plan generation. Benyamin et al. (2025) describe a similar division of labor in *Planning Copilot*, where the chatbot accesses external planning tools for syntax checking, planner selection, plan generation, validation, and simulation.

2.4 Model Context Protocol

Model Context Protocol (MCP) is a protocol for connecting AI-based client applications to external capabilities exposed by another process or service. In practice, it follows a client-server architecture in which a client requests tool use, and a server exposes those tools and returns structured results. It allows software to present operations as callable tools through a standard interface, rather than requiring every client to implement a different custom integration for each back-end system. This is especially useful in AI-assisted systems, where a model-driven client may need to access many specialized capabilities while remaining separate from the implementation of those services. That use of MCP appears directly in recent planning work: Benyamin et al. (2025) use MCP to connect a chatbot to planning tools in *Planning Copilot*, Jobs et al. (2025) use MCP as a standardized tool interface in their Blocksworld benchmark, and Göbel et al. (2026) expose planning operations as MCP tool calls in their PyPDDLEngine system.

In this project, MCP serves as the integration layer between the Discord bot and the planning back ends. The bot maintains live MCP connections to both `planning-as-a-service` and the `l2p_mcp` server during runtime. Through those connections, the bot can access planning-related capabilities while remaining focused on conversation flow and user interaction. This improves modularity because planning-model updates, validation, and solver access can be handled by separate services instead of being embedded directly in the bot. It also improves extendibility, since additional planning capabilities can be exposed as MCP tools without requiring the main user-facing application to be redesigned.

3 System Overview

3.1 Overall Architecture

At a high level, the system follows a layered architecture. Discord provides the user-facing entry point, MCP provides the integration boundary, and the two back-end services (`planning-as-a-service` and `l2p_mcp`) provide the planning-specific functionality.

The end-to-end flow is as follows: a user issues a request through the Discord bot, the bot routes planning-related requests through its MCP client layer, and the appropriate back end service handles the task. Requests involving planning-model construction or revision through natural language are directed to `l2p_mcp`, while requests involving solving or validation are directed to `planning-as-a-service`. The resulting artifacts or outputs are then returned through the bot to the user.

3.2 `IPA-Discord-Bot`

The `IPA-Discord-Bot` repository is the user-facing interface of the project. Instead of requiring users to work directly with planning files, service endpoints, or command-line tools, it lets the workflow take place through natural language interaction on Discord. The main commands include `!plan`, `!domain`, and `!problem` for planning requests, along with `!show`, `!edit`, `!undo`, `!files`, and `!explain` for inspecting and revising artifacts. The user may also decide to validate the artifacts through the following set of commands: `!validate`, `!validate_domain`, `!validate_task`, and `!validate_plan`. Further information on these commands can be found in *Appendix: Discord Bot Commands*.

The bot also includes database management through sqlite3, which stores the current domain, problem, and plan for either an individual user or a shared channel context, so later commands can continue from the existing state. It is also responsible for conversation logging, specifically storing messages under user_id and channel_id keys to support per-user workflows. When the `!collab` command is called, the bot toggles the LLM context to the channel's logged messages instead of individual user messages for collaborative planning.

3.3 `planning-as-a-service`

`planning-as-a-service` is the main solver-oriented back end in the system. It provides an extendable API for deploying and running planning tools locally through remote endpoints. In the context of this project, it serves as the component responsible for exposing planning functionality

beyond the Discord interface itself. Its existing API supports package-based service calls for planner operations. The main addition in this project was adding its MCP layer and deploying to its persistent remote server, which makes those capabilities available to external MCP clients in a tool-oriented form.

This repository was especially relevant because it formed the bridge between formal planning services and the rest of the assistant architecture. The `planning-as-a-service` MCP server can expose planning tools based on available package manifests from `planutils`, which is a toolkit that exposes various planning utilities such as solvers, validation, and PDDL syntax checking. For MCP, the remote endpoint is available at `https://solver.planning.domains/mcp`. This means that planner-related capabilities could be accessed through MCP rather than through direct, bot-specific integrations.

3.4 `l2p_mcp`

The `l2p_mcp` repository is a standalone MCP server that wraps deterministic parts of the external `L2P` toolkit for human-in-the-loop planning workflows. Its role is different from `planning-as-a-service`: rather than focusing on solver execution, it focuses on planning-model construction and revision. This component is important because it supports incremental editing of planning models instead of forcing the system to regenerate an entire domain or problem from scratch after each user request. The server exposes two main tools, `update_domain` and `update_task`, and handles parsing, merging, and normalization internally. It can also be run as a long-lived Dockerized service over HTTP, which makes it suitable for integration with the Discord bot through MCP. Within the overall architecture, `l2p_mcp` therefore acts as the structured modeling layer of the system, complementing the solver-oriented role of `planning-as-a-service`.

3.5 Interactions Between Repositories

These repositories form a layered workflow rather than three independent systems. The bot is the user-facing entry point that handles module interaction and orchestration, `l2p_mcp` handles structured model updates and creation through natural language requests, and `planning-as-a-service` provides solving and validation services. Keeping these three codebases separate reduced coupling and made it easier to test, deploy, and extend each part individually, without forcing all changes into one codebase.

4 Approach and Implementation

4.1 Design Approach

The implementation strategy was to keep the Discord bot as the orchestration layer and move planning-specific logic into MCP-accessible services. The bot handled user interaction, request routing, and persistent state, while execution and model-transformation logic were kept outside of it. This kept the chat interface separate from the planning back ends and made the system easier to test and modify.

The design also split planning work into execution-oriented tasks and modeling-oriented tasks. The MCP interfaces were kept intentionally high level, so the bot only needed to choose the right service while each back end handled its own parsing, validation, execution, and result formatting. Combined with stored artifact state, this supported an incremental workflow in which users could refine models, validate them, and solve them without restarting the pipeline at each step.

4.2 Work in `planning-as-a-service`

Work in `planning-as-a-service` focused on turning the existing planning API into a back end that could be accessed by MCP clients. Rather than requiring the Discord bot to call solver-specific HTTP routes directly, the approach introduced an MCP server to wrap around `planning-as-a-service` so that planning capabilities could be exposed as tools with a consistent calling interface. This kept the planning logic inside the service layer while giving the rest of the system cleaner integration.

The main implementation step was the addition of the `planning-as-a-service` MCP server in `server/mcp`. Its wrapper code connects to the existing `planning-as-a-service` API, submits jobs, polls for completion, and returns normalized tool responses. The wrapper also dynamically registers MCP tools based on the available package manifests sourced from `planutils`. This allows the exposed tool surface to reflect the planners and utilities already installed instead of relying on one-off handwritten wrappers.

Another important part of the `planning-as-a-service` work was adding support for plan validation and syntax checking. The services include both the `val` package and the `pddl` package from `planutils`. In particular, the `pddl` tool had to be added to `planutils` before integrating it with `planning-as-a-service`. These additions made both packages accessible from the `planning-as-a-service` API endpoint. Corresponding MCP tools in the repository also gave access to tools such as `paas_val_validate` and `paas_pddl_validate_plan`. This was important for the larger project because planning workflows need more than just solver execution. Users also need ways to validate plans and catch malformed PDDL before or after attempting to solve a

task.

Finally, the `planning-as-a-service` MCP layer was designed to support both local and hosted use. The repository includes configuration for streamable HTTP transport, and `https://solver.planning.domains/mcp` as the remote endpoint used for hosted access. This made `planning-as-a-service` not just a local development dependency, but a reusable planning service that other components in the project, especially the Discord bot, could reach through MCP. This addition became not only useful to this particular project, but also to other potential users of `planning-as-a-service` who want to access its capabilities through an MCP-oriented interface, particularly for accessing planning services through AI-based applications.

4.3 Work in `l2p_mcp`

The work in `l2p_mcp` focused on exposing the planning-model editing capabilities of the external `L2P` toolkit through a dedicated MCP server. Unlike `planning-as-a-service`, which is solver-oriented, `l2p_mcp` handles the construction and revision of planning domains and tasks through natural language input. Keeping it separate made the overall architecture more modular and allowed modeling support to evolve independently from solving support.

The server exposes two high-level tools, `update_domain` and `update_task`. These tools accept constrained updates, parse the requested changes, merge them into the current planning state, and generate updated PDDL fragments when enough information is available. Helper logic in `server_helpers.py` handles parsing, normalization, and merging so repeated or partial updates remain stable. These helpers all access capabilities from the `L2P` toolkit. This module was important because the larger system depends on being able to revise only part of a domain or task without discarding the rest of the user's work, which is the heart of human-in-the-loop development.

Another important aspect of this repository was making the editing workflow deterministic after the client had decided what kind of update to send. The client-side model decides whether a request belongs to the domain or the task, and the MCP server then handles parsing, merging, and artifact generation. The repository also includes direct and MCP integration tests, along with Docker support for long-running HTTP deployment.

4.4 Work in `IPA-Discord-Bot`

MCP client integration played a central role in the implementation of `IPA-Discord-Bot`. The bot handles user interaction, request flow, and stored workflow state while accessing planning-specific capabilities through MCP calls to the two back end services. Commands such as `!plan`, `!domain`, and `!problem` are routed through this client layer, which then calls `l2p_mcp` for modeling updates

and `planning-as-a-service` for validation and solving. More detail appears in the *Planning Commands* entry of *Appendix: Discord Bot Commands* and in *Appendix: MCP Tools*.

The bot also keeps track of workflow state across multiple interactions. It stores the current domain, problem, and plan so users can continue from the same artifacts, and saved sessions can be retrieved later. This lets the bot focus on conversation logging, model selection, collaboration mode, and workflow continuity while the back-end services handle the planning logic.

This repository also required implementation work around conversational routing rather than only command handling. The bot supports normal chat messages, explicit planning commands, artifact inspection and revision, validation commands, and MCP tool inspection within the same Discord interface. To make that work reliably, the bot had to distinguish between ordinary conversation and requests that should trigger planning-related workflows. In practice, this meant combining lightweight LLM-based interpretation with more structured command-driven paths so that natural-language interaction remained flexible without losing access to deterministic planning tools.

Another important part of the bot implementation was persistence and shared context management. The bot uses SQLite to store message history, current planning artifacts, saved sessions, and per-user settings such as selected models and provider keys. This allowed the Discord interface to support workflows that extend over time instead of treating each request as isolated. The same persistence layer also made collaborative features possible, since the `!collab` mode could switch the effective context from an individual user's history to a shared channel-level context. As a result, the bot was able to function not just as a prompt front end for planning services, but as the main coordination layer that preserves state and keeps multi-step interaction manageable for users.

4.5 Deployment and Infrastructure

Deployment was important because the project was built as a set of cooperating services rather than a single local program. For the workflow to be usable in practice, the solver-oriented `planning-as-a-service` MCP server, the modeling-oriented `l2p_mcp` server, and the Discord bot all needed to be available as stable components in the same overall pipeline.

Within that deployment structure, each component had a different role. The `planning-as-a-service` MCP server was deployed to a remote server and exposed through the hosted endpoint `https://solver.planning.domains/mcp`, making planning and validation capabilities reachable beyond a single development machine. The `l2p_mcp` service was dockerized so it could run persistently as the domain and task-editing back end used for iterative planning-model updates. `IPA-Discord-Bot` was also dockerized, allowing the user-facing front end to run as a long-lived service that could stay connected to the two MCP back ends.

Taken together, these deployment choices meant the final system was not just a set of related

repositories, but a usable service architecture: one remotely available planning back end, one dockerized modeling back end, and one dockerized user-facing interface connected through MCP.

4.6 Key Challenges and Design Tradeoffs

One of the main issues while creating the bot was its dependency on LLMs for interpreting user requests and generating updates. Although the goal was to balance flexibility and reliability, the bot's reliance on LLMs for core planning tasks subjected it to occasional interpretation and output inconsistencies. While the bot often understood user requests, it would sometimes produce plans that deviate from the user's intent. Although it was possible to implement deterministic measures to increase reliability, it would trade off some of the flexibility that the bot was designed to provide.

Another challenge was the implementation of `L2P` in the integration between the bot and the back end MCP server. During the process of development, `L2P` was added on both ends, since it was unclear where the structured planning-model updates should be generated. The final design used the LLM from the Discord bot to orchestrate deterministic update tools in `l2p_mcp` such as `update_domain` and `update_task`. This design choice was important because it served to bridge both parts of the system. The bot's LLM could focus on interpreting user requests and generating structured update payloads, while the MCP server could handle the more deterministic parsing, merging, and PDDL generation.

5 Results

5.1 Completed Functionality

The system supports an end-to-end planning workflow inside Discord. Users can issue planning requests in natural language, generate PDDL domain and problem artifacts, submit those artifacts for solving, and receive the resulting outputs back through the bot. The bot also supports direct planning requests from attached PDDL files, which means the workflow can begin either from natural language or from already prepared planning artifacts.

The implemented functionality also includes iterative artifact management. Through commands such as `!show`, `!edit`, `!undo`, `!files`, and `!explain`, users can inspect and revise the current working domain, problem, and plan without leaving Discord. This is supported by the `l2p_mcp` back end, which exposes high-level domain and task update tools that allow partial planning-model changes to be merged into the current state rather than forcing full regeneration after every request.

On the execution side, the system supports solving, validation, and syntax checking through the `planning-as-a-service` MCP server. The bot configuration and help output show working

support for plan generation, VAL-based validation, and PDDL syntax validation of domains, tasks, and plans. In addition, the system supports MCP tool discovery commands, persistent or shared workflow state in Discord, per-user model selection, encrypted provider-key storage, and collaboration-oriented channel features. Taken together, this means the final system is not only a proof of concept for isolated components, but a functioning integration of a conversational interface, a planning-model editing service, and a planning-and-validation back end.

5.2 Demonstration Scenarios

One demonstration scenario was the recorded walkthrough linked in the *Video Demonstration* entry of *Appendix: Supplementary Materials*. The video presented the intended end-to-end workflow of the system: a user interacting with the Discord bot, generating and updating planning artifacts, and then using the connected planning services to obtain solver and validation feedback. This made the video a compact demonstration of how the repositories worked together in practice.

Another demonstration scenario was the public presentation of the project poster and working bot at the 2026 Creative Computing Showcase hosted by the Queen's School of Computing. At that event, we presented the poster shown in the `IPA-Discord-Bot` *Poster* entry of *Appendix: Supplementary Materials*. The event was open to drop-in visitors and centered on hands-on demonstrations, which made it a good setting for talking with attendees about the project and letting visitors test the bot's features live. During these live interactions, the bot functioned with minimal visible error. The most notable problems appeared when generating PDDL problems at significant scale, which is consistent with the broader survey conclusion from Tantakoun et al. (2025) that LLM-supported planning workflows still face important difficulties on more complex structured planning tasks.

Weekly meetings with Professor Christian Muise and supervisor Marcus Tantakoun also served as demonstration scenarios throughout development. These meetings created regular checkpoints where the current state of the system could be shown, recent changes could be discussed, and feedback could be incorporated into later iterations.

5.3 Testing and Validation

The evidence for correctness came from several layers. In `l2p_mcp`, the repository includes both direct offline tests and MCP integration tests. These tests cover domain and task creation, incremental updates, multi-action parsing, file-based inputs, error handling, and the ability to expose and call the two high-level MCP tools through a temporary client session. This gave strong evidence that the planning-model editing service behaved consistently both as a local Python component and as an MCP server.

Additionally, `planning-as-a-service` includes explicit MCP testing procedures and supporting scripts. Its testing workflow covers both local MCP testing and hosted remote MCP testing against `https://solver.planning.domains/mcp`. The dynamic MCP test script exercises solver calls, VAL-based validation, and PDDL syntax validation, including both positive and negative cases, while a separate manifest-check script verifies that dynamically generated MCP tool signatures remain aligned with the underlying package manifest.

For the `IPA-Discord-Bot` repository, end-to-end Discord behavior was validated more through manual testing than through large automated suites. The most important settings were the recorded demo video, the Creative Computing showcase, and the weekly meetings with Professor Christian Muise and Marcus Tantakoun, all of which tested both individual components and the integrated workflow. Additionally, the bot itself exposes validation commands for domains, tasks, plans, and domain-problem-plan consistency, so correctness checks were also built directly into the user workflow.

5.4 Assessment Against Objectives

By the end of development, the project successfully implemented the `IPA-Discord-Bot` with all of the core functionalities described in the initial stages of planning. The bot can smoothly converse with the user through a natural-language conversation workflow, and can generate, revise, and validate plans based on user requests. It was also properly organized into a layered architecture system with the bot acting as the user interface, and the two back end services providing plan editing and solving functionalities. Conversation histories are also logged and stored for later retrieval when the user wishes to resume conversations with context from previous sessions. The implementation of the `!collab` command also allows multiple users to work together on the same planning models. Additional functionalities include features such as per-user model selection, thread creation, secure key sharing, and more that allow the bot to be more flexible in Discord. Overall, the project was completed with the core objectives met, and multiple additional features implemented to improve user experience and make the system more robust.

6 Open Problems and Future Work

6.1 Current Limitations

Despite the functionality of the project in its current state, there is still room to improve reliability. The workflow of the bot remains dependent on the quality of large language model outputs, so incomplete updates or revisions that do not fully match the user's intent are still possible. This

limitation aligns with two recurring points in the literature: the Tantakoun et al. (2025) survey repeated findings that LLMs struggle with long-horizon planning, and Göbel et al. (2026) report that even an agentic LLM planning setup gains only modestly from step-wise PDDL interaction because the agent still has to evaluate largely self-assessed planning feedback.

To keep the bot responsive, conversation logging and artifact history are bounded in size, and the current workflow state is only preserved across sessions when it is explicitly saved. There are also usability and deployment limits. The bot depends on configured MCP endpoints and provider keys, and it assumes that the modeling and solving back ends are reachable. If either of the MCP components are unavailable, the bot cannot function correctly. While Discord makes the workflow more accessible than direct PDDL authoring, it still requires that users have some understanding of domains, problems, plans, and validation steps.

6.2 Future Extensions

One important direction for future work would be strengthening persistence and recovery. The bot already stores working artifacts and supports revision commands, so a natural next step would be preserving workflow state more automatically, improving artifact version tracking, and making it easier to resume work without depending on manual session-saving behavior.

Another important extension would be improving the human-in-the-loop workflow itself. The system already supports inspection, revision, undo, and validation, but it could provide better summaries of artifact changes, clearer explanations after validation failures, and a more reliable repair loop around the current beta `!autovalidate` workflow.

Future work could also expand the range of MCP-exposed tools. Additional planners, richer validation capabilities, and tools for analyzing plan quality could fit naturally into the current architecture. This matches the broader direction of MCP-based planning systems such as *Planning Copilot* (Benyamin et al., 2025) and the benchmark environment proposed by Jobs et al. (2025).

Finally, there is room for continued work on deployment and usability. A more unified deployment process for all components would make the system easier to run and maintain over longer periods, especially since the current implementation depends on multiple configured services and environment settings. The Discord interface could also be extended with richer interaction patterns, such as clearer revision history presentation, improved artifact comparison, and more guided workflows for first-time users. In this way, future work would not only improve the technical reliability of the planning pipeline, but would also make the system easier for users to understand and use in practice.

7 Conclusion

The Interactive Planning Agent set out to make automated planning more accessible through a natural language interface. Rather than expecting users to work directly with separate planning tools, files, and services, the goal was to support a workflow in which planning artifacts could be created, revised, solved, and validated with natural language through Discord. To support that goal, the final system combined a Discord bot front end with two MCP server back ends: one for planning-model editing and one for solving and validation.

Now that the project is complete, that integrated workflow is functioning. The bot accepts natural-language requests or attached planning files, maintains a working planning state, supports iterative revision of domains and problems, and calls external planning services for solving and validation. The system is also structured in a way that kept its major responsibilities separate: `IPA-Discord-Bot` handles interaction and workflow management, `l2p_mcp` handles structured domain and task updates, and `planning-as-a-service` handles plan generation and validation. This separation makes the project more modular and easier to extend than a single monolithic bot implementation would have been.

More broadly, the project shows that MCP can serve as a useful interface layer for combining conversational systems with automated planning services. It also highlights both the strengths and the current limitations of LLM-supported planning workflows. The system performed well in live demonstrations and supported meaningful interaction, while also showing that large-scale structured PDDL generation remains challenging. Even with those limits, the final result provides a working foundation for interactive planning in Discord and a useful starting point for future improvements.

References

[1] Benyamin, Y., Mordoch, A., Shperberg, S. S., & Stern, R. (2025). *Toward PDDL planning copilot*. arXiv. `https://doi.org/10.48550/arXiv.2509.12987`

[2] Göbel, K., Lorang, P., Zips, P., & Glück, T. (2026). *Agentic LLM planning via step-wise PDDL simulation: An empirical characterisation*. arXiv. `https://doi.org/10.48550/arXiv.2603.06064`

[3] Jobs, N., Vieira da Silva, L. M., Somashekaraiah, J., Weigand, M., Kube, D., & Gehlhoff, F. (2025). *Benchmark for planning and control with large language model agents: Blocksworld with model context protocol*. arXiv. `https://doi.org/10.48550/arXiv.2512.03955`

[4] Tantakoun, M., Zhu, X., & Muise, C. (2025). *LLMs as planning formalizers: A survey for leveraging large language models to construct automated planning models*. arXiv. https://doi.org/10.48550/arXiv.2503.18971

[5] Wang, J., Peng, R., Chen, T., Zeng, C., Wang, W., Huang, S., & Lipovetzky, N. (2025). *PDDL-GenAI – A plugin for LLM-based support in planning domain modeling*. In *ICAPS-25 System Demonstrations*. https://icaps25.icaps-conference.org/program/demos-pdfs/ICAPS25-Demo_paper_1.pdf

A Appendix: Project Repositories

A.1 Foundation Repositories

- `planning-as-a-service`: The main planning-service back end used in the project. This repository provided the existing planner API and service architecture that the project extended with MCP support. Official repository: https://github.com/AI-Planning/planning-as-a-service
- `planutils`: The planning package ecosystem used by `planning-as-a-service` to expose installed planners and related tools. It was also relevant to this project because validation and PDDL syntax-checking support depended on packages installed through `planutils`. Official repository: https://github.com/AI-Planning/planutils
- `L2P`: The Language-to-Plan toolkit used as the foundation for the `l2p-mcp` server. Its domain and task building utilities were wrapped through MCP to support structured planning-model editing. Official repository: https://github.com/AI-Planning/l2p

A.2 Repositories Created for this Project

- `l2p_mcp`: The MCP server developed in this project to expose selected `L2P` capabilities through high-level tools for generating or updating planning domains and tasks through natural language. Repository used in this project: https://github.com/adam-neto/l2p_mcp
- `IPA-Discord-Bot`: The user-facing Discord bot repository used in this project. It provided the conversational interface and was extended to access planning functionality through MCP-connected back ends. Repository used in this project: https://github.com/jawadahmed02/IPA-Discord-Bot

B Appendix: Discord Bot Commands

B.1 Planning Commands

- `!help`: Show the built-in help message with a short description of each command.
- `!plan <request>`: Solve a planning request from natural language, or from attached domain and problem PDDL files.
- `!plan`: Re-solve using the current stored domain and problem in the current channel.
- `!domain <request>`: Generate a planning domain from a natural-language request and return the resulting domain PDDL.
- `!problem <request>`: Generate a planning problem from a natural-language request and return the resulting problem PDDL.

B.2 Human-in-the-Loop Editing Commands

- `!files`: Send the current stored domain, problem, and plan as files.
- `!show <domain|problem|plan>`: Show the current working artifact in chat.
- `!explain <domain|problem|plan>`: Explain the current working artifact in natural language.
- `!edit <domain|problem|plan> <instruction>`: Revise one current artifact while preserving the rest of the workflow state.
- `!undo <domain|problem>`: Restore the previous version of one user-authored artifact.

B.3 Validation Commands

- `!validate`: Validate that a plan fits a domain and problem with VAL, using attachments or the last successful `!plan` output.
- `!validate_domain`: Validate a domain PDDL file with the `planning-as-a-service` domain validation tool.
- `!validate_task`: Validate a domain/problem pair with the `planning-as-a-service` task validation tool.

- `!validate_plan`: Validate a domain/problem/plan triple with the `planning-as-a-service` plan validation tool.
- `!autovalidate`: Beta command that loops domain and problem repairs against VAL until the current plan passes or the retry limit is reached.

B.4 MCP Tool Inspection Commands

- `!tools`: List the MCP tools currently exposed by the connected planning servers.
- `!paastools`: List only the tools currently exposed by the connected PaaS MCP server.

B.5 Chat and Thread Commands

- `!thread [topic]`: Create a private thread in the current server channel for chatting with the bot.
- `!chat`: Toggle normal-message bot chat on or off for the current user in the current channel.
- `!collab`: Toggle shared collaboration mode for the current channel or thread.

B.6 Bot Setting Commands

- `!share`: Toggle whether other users may use your saved provider key when they do not have one.
- `!save`: Save the current bot session so its conversation log is kept after restart.
- `/models`: Show the available LLM model IDs you can choose from.
- `/use <model_id>`: Set which model the bot should use for your requests.
- `/setkey <provider> <api_key>`: Save your provider API key for `openai`, `gemini`, or `anthropic`.

C Appendix: MCP Tools

C.1 `planning-as-a-service` Tools

- `paas_list_packages`: Returns the installed package and service manifests exposed by the `planning-as-a-service` MCP server.

- `paas_[planner]_solve`: Set of tools that submit a domain and problem to a chosen planner through `planning-as-a-service` and return the resulting plan output.
- `paas_val_validate`: Runs VAL-based validation to check whether a generated plan is a valid solution to a given domain and problem pair.
- `paas_pddl_validate_domain`: Checks the syntax of a PDDL domain file.
- `paas_pddl_validate_task`: Checks the syntax and structure of a PDDL domain and problem pair.
- `paas_pddl_validate_plan`: Checks the syntax of a PDDL domain, problem, and plan combination.

C.2 `L2P` Tools on `l2p_mcp`

- `update_domain`:
  - Creates or updates planning-domain state from model-written text.
  - Supports high-level domain elements such as types, constants, predicates, functions, and actions.
  - Can start from an empty domain or merge changes into an existing one.
  - Can infer requirements and generate updated domain PDDL when enough information is provided.
- `update_task`:
  - Creates or updates planning-task or problem state from model-written text.
  - Supports core task elements such as objects, initial conditions, and goals.
  - Can start from an empty task or merge changes into an existing one.
  - Can generate a new or updated problem PDDL when enough information is provided.

D Appendix: Supplementary Materials

D.1 Video Demonstration

Project demonstration video: `https://www.loom.com/share/7c9f8f98e8f44776b55249da51d1f004`

D.2 `IPA-Discord-Bot` Poster

Interactive Planning Agent Discord Bot

Project by Adam Neto and Jawad Ahmed
Supervised by Professor Christian Muise and Marcus Tantakoun
Completed under **MuLab** at Queen's University in Kingston

Motivation

- AI planning is powerful but requires expert-level formal modeling (e.g., PDDL), which limits accessibility
- While LLMs enable natural language interaction, they lack reliability for structured planning tasks
- This project bridges that gap by combining LLMs with formal planning and human-in-the-loop collaboration

Approach

- Develop an interactive Discord-based system that translates natural language into formal planning models
- Use LLMs, Planning-as-a-Service, and Language-to-Plan to produce PDDL Domains, Problems, and Plans
- Users iteratively refine these models through human-in-the-loop feedback, while planning tools validate/solve

Example PDDL Domain, Problem, and Plan

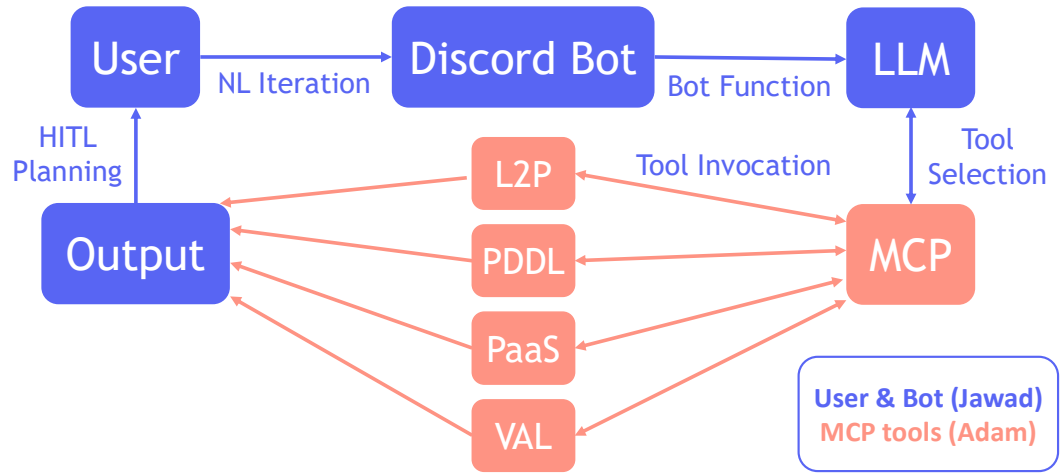
```
(define (domain move)
  (:predicates (atA) (atB))
  (:action go
    :precondition (atA)
    :effect (and (not (atA)) (atB))
  )
)
```

```
(define (problem p1)
  (:domain move)
  (:init (atA))
  (:goal (atB))
)
```

(go)

Result: Move from A to B

System Architecture



Methods

- **Discord Bot Interface:** Users submit planning requests in natural language through Discord
- **Language Processing:** An LLM interprets the request and extracts task information
- **Planning Model Generation:** L2P interprets the request and extracts planning domain and task information
- **Plan output:** The generated plan is returned to the user through the discord bot

MCP Tools

- **L2P:** Generates PDDL including a Domain and Task
- **PDDL:** Verifies PDDL syntax after L2P PDDL generation
- **Planning as a Service:** Publicly accessible API/MCP endpoint to generate Plans given Domain and Problem
- **VAL:** Tool to validate that a PDDL Plan is a viable solution to a given PDDL Domain and Problem

Project GitHub: https://github.com/jawadahmed02/IPA-Discord-Bot. GitHub usernames: @adam-neto, @jawadahmed02